STAT 432 Homework 03

Conner Jensen (connerj2)

Contents

Question 1: Build an `explain-to-me` skill **1**

Question 2: Ridge regression with nearly collinear predictors **2**

(a) . . . 2

(b) . . . 2

(c) . . . 3

Question 3: What does ridge regression shrink? **3**

(a) . . . 3

(b) . . . 4

(c) . . . 4

Question 4: Ridge regression and optimization **4**

(a) . . . 4

(b) . . . 5

(c) . . . 5

Question 5: Selecting the ridge penalty for real-estate prediction **5**

(a) . . . 6

(b) . . . 7

(c) . . . 8

Reference **9**

`explain-to-me` skill **9**

Question 1: Build an `explain-to-me` skill

I wrote a short skill `explain-to-me` and installed it at `.cursor/skills/explain-to-me/SKILL.md`. It is meant to explain a homework question (goal, required work, constraints, lecture pointer) and not to solve it.

I tested it on Question 2. The skill's reading of that question is:

- **Goal.** See how ridge behaves when two predictors are nearly copies of each other.
- **Required work.** Draw one standardized design, report $\mathrm{Cor}(X_1, X_2)$ and the extreme singular values, then repeat y, y^* 200 times and summarize $\widehat{\beta}_1$, $\widehat{\beta}_2$, their sum and difference, and train/test MSE at $\lambda \in \{0, 0.02, 0.2\}$.
- **Constraints.** Seed `43231`; lecture scaling (mean 0, average squared value 1); intercept unpenalized; $\lambda = 0$ is OLS.
- **Lecture.** Ridge Regression: Stability Through Shrinkage (Week 3 notes).

The full `SKILL.md` is appended at the end of this report.

Question 2: Ridge regression with nearly collinear predictors

```
set.seed(43231)
n <- 100
L <- rnorm(n)
X_raw <- cbind(
  L + 0.04 * rnorm(n),
  L + 0.04 * rnorm(n),
  rnorm(n),
  rnorm(n)
)
X <- standardize(X_raw)
sv <- svd(X, nu = 0, nv = 0)$d
c(cor12 = cor(X[, 1], X[, 2]),
  d_max = max(sv),
  d_min = min(sv))
```

```
##      cor12      d_max      d_min
##  0.9977057 14.1414804  0.4753575
```

(a)

The first two columns have correlation about 0.998. The singular values of $\mathbf{X}$ run from about 14.14 down to 0.48. After scaling, each column has $\|\mathbf{x}_j\|_2^2 = n$, so the shared latent factor L produces a large direction $\mathbf{x}_1 + \mathbf{x}_2$ and a tiny direction $\mathbf{x}_1 - \mathbf{x}_2$. OLS divides by d_j^2 in that contrast direction, so $\widehat{\beta}_1 - \widehat{\beta}_2$ is much noisier than $\widehat{\beta}_1 + \widehat{\beta}_2$.

(b)

```
beta <- c(1.5, 1.5, 1, 0)
mu <- as.numeric(X %*% beta)
nsim <- 200
lams <- c(0, 0.02, 0.2)
store <- array(NA_real_, c(nsim, length(lams), 6))
for (s in seq_len(nsim)) {
  y <- mu + rnorm(n)
  ystar <- mu + rnorm(n)
```

```
  ybar <- mean(y)
  ytilde <- y - ybar
  for (j in seq_along(lams)) {
    b <- ridge_beta(X, ytilde, lams[j])
    yhat <- ybar + as.numeric(X %*% b)
    store[s, j, ] <- c(b[1], b[2], b[1] + b[2], b[1] - b[2],
                       mean((y - yhat)^2), mean((ystar - yhat)^2))
  }
}

q2_summary <- do.call(rbind, lapply(seq_along(lams), function(j) {
  data.frame(
    lambda = lams[j],
    mean_b1 = mean(store[, j, 1]), sd_b1 = sd(store[, j, 1]),
    mean_b2 = mean(store[, j, 2]), sd_b2 = sd(store[, j, 2]),
    mean_sum = mean(store[, j, 3]), sd_sum = sd(store[, j, 3]),
    mean_diff = mean(store[, j, 4]), sd_diff = sd(store[, j, 4]),
    train_mse = mean(store[, j, 5]),
    test_mse = mean(store[, j, 6])
  )
}))
q2_summary
```

```
##   lambda  mean_b1      sd_b1  mean_b2      sd_b2 mean_sum     sd_sum
## 1   0.00 1.495589 1.54394848 1.509765 1.54858847 3.005354 0.09510778
## 2   0.02 1.486994 0.16151331 1.489176 0.16591120 2.976171 0.09416526
## 3   0.20 1.367908 0.04570277 1.368693 0.04733702 2.736601 0.08646679
##      mean_diff    sd_diff train_mse test_mse
## 1 -0.014175478 3.09107762 0.9647563 1.062452
## 2 -0.002181997 0.31362247 0.9746809 1.049262
## 3 -0.000785071 0.03438848 1.0767603 1.133484
```

(c)

At $\lambda = 0$, $\widehat{\beta}_1$ and $\widehat{\beta}_2$ each have SD about 1.54, and the contrast SD is about 3.09. The sum $\widehat{\beta}_1 + \widehat{\beta}_2$ stays near 3 with SD only 0.10. A small penalty $\lambda = 0.02$ barely moves the sum (mean 2.98, SD 0.09) but cuts the contrast SD to 0.31. Average test MSE falls slightly (1.062 to 1.049). A large penalty $\lambda = 0.2$ over-shrinks the sum toward 2.74 and raises test MSE to 1.13.

Ridge damps the poorly determined contrast without changing the well-determined sum much, so the fitted vector $\mathbf{X}\widehat{\boldsymbol{\beta}}$ moves much less than the separate coefficients. That is a bias–variance trade-off on the small singular direction, not a large change in the fitted mean.

Question 3: What does ridge regression shrink?

(a)

Differentiate

$$L_\lambda(\boldsymbol{\beta}) = \frac{1}{2n}\|\widetilde{\mathbf{y}} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \frac{\lambda}{2}\|\boldsymbol{\beta}\|_2^2$$

to get

$$\nabla L_\lambda(\boldsymbol{\beta}) = -\frac{1}{n}\mathbf{X}^\mathsf{T}(\widetilde{\mathbf{y}} - \mathbf{X}\boldsymbol{\beta}) + \lambda\boldsymbol{\beta}.$$

Set the gradient to zero and multiply through by n:

$$(\mathbf{X}^\mathsf{T}\mathbf{X} + n\lambda\mathbf{I}_p)\widehat{\boldsymbol{\beta}}_\lambda = \mathbf{X}^\mathsf{T}\widetilde{\mathbf{y}}.$$

If $\lambda > 0$, then $n\lambda\mathbf{I}_p$ is positive definite, so $\mathbf{X}^\mathsf{T}\mathbf{X} + n\lambda\mathbf{I}_p$ is positive definite even when $\mathbf{X}^\mathsf{T}\mathbf{X}$ is singular or $p > n$. A positive definite matrix is invertible, so the solution is unique.

(b)

Write $\mathbf{X} = \mathbf{U}_r\mathbf{D}_r\mathbf{V}_r^\mathsf{T}$. Then

$$\widehat{\boldsymbol{\beta}}_\lambda = \mathbf{V}_r(\mathbf{D}_r^2 + n\lambda\mathbf{I}_r)^{-1}\mathbf{D}_r\mathbf{U}_r^\mathsf{T}\widetilde{\mathbf{y}},$$

and the fitted mean is

$$\mathbf{X}\widehat{\boldsymbol{\beta}}_\lambda = \mathbf{U}_r\,\mathrm{diag}\left(\frac{d_j^2}{d_j^2 + n\lambda}\right)\mathbf{U}_r^\mathsf{T}\widetilde{\mathbf{y}}.$$

The factor on the jth left singular direction is $\rho_j(\lambda) = d_j^2/(d_j^2 + n\lambda)$.

```
n <- 100
lambda <- 0.01
d <- c(10, 2, 0.2)
rho <- d^2 / (d^2 + n * lambda)
c(rho1 = rho[1], rho2 = rho[2], rho3 = rho[3],
  df_eff = 1 + sum(rho))
```

```
##       rho1       rho2       rho3     df_eff
## 0.99009901 0.80000000 0.03846154 2.82856055
```

The shrinkage factors are 0.990, 0.800, and 0.0385. Including the unpenalized intercept, $\mathrm{df}_{\mathrm{eff}}(0.01) = 2.829$.

(c)

The $d_3 = 0.2$ direction is shrunk the most ($\rho_3 \approx 0.038$). OLS is most variable there because the variance factor is on the order of $1/d_j^2$. Shrinking that component cuts prediction variance. The same shrinkage creates substantial bias if the true signal $\boldsymbol{\mu}$ has a large coordinate along that left singular vector.

Question 4: Ridge regression and optimization

(a)

```
a0 <- c(9, 0.01)
lambda <- 0.09
al <- a0 + lambda
c(cond_A0 = max(a0) / min(a0),
  cond_Alambda = max(al) / min(al))
```

```
##      cond_A0 cond_Alambda
##        900.0         90.9
```

$\kappa(\mathbf{A}_0) = 900$ and $\kappa(\mathbf{A}_\lambda) = 90.9$. Adding $\lambda \mathbf{I}_p$ lifts the tiny curvature from 0.01 to 0.10, so the quadratic bowl is much less elongated. Gradient steps then move at more comparable speeds in the two eigendirections.

(b)

The ridge gradient is $\nabla L_\lambda(\boldsymbol{\beta}) = \mathbf{A}_\lambda \boldsymbol{\beta} - \mathbf{X}^\mathsf{T}\tilde{\mathbf{y}}/n$, and $\mathbf{A}_\lambda \widehat{\boldsymbol{\beta}}_\lambda = \mathbf{X}^\mathsf{T}\tilde{\mathbf{y}}/n$, so

$$\mathbf{e}^{(k+1)} = \big(\mathbf{I} - \eta \mathbf{A}_\lambda\big)\mathbf{e}^{(k)}.$$

In an eigendirection with unpenalized eigenvalue a, the factor is $1 - \eta(a + \lambda)$.

Every factor has absolute value less than 1 iff $0 < \eta < 2/\nu$ for every eigenvalue ν of $\mathbf{A}_\lambda$. The tightest bound uses the largest eigenvalue $M = 9.09$:

$$\eta \in (0, 2/M) = (0, 0.2200).$$

- For $\eta = 1/M$, the factors are $1 - \eta \cdot 9.09 = 0$ and $1 - \eta \cdot 0.10 \approx 0.989$. The high-curvature direction converges immediately; the flat direction is slow.
- For $\eta = 2.1/M$, the factor on the M direction is -1.1. If the initial error has a component there, that component grows and the method does not converge.

(c)

A large λ makes $\mathbf{A}_\lambda$ well conditioned and can make GD fast, but it also shrinks $\widehat{\boldsymbol{\beta}}_\lambda$ toward zero. Penalty choice is a statistical bias–variance decision for prediction, not a device for speeding an optimizer.

Question 5: Selecting the ridge penalty for real-estate prediction

```r
re <- read.csv("data/realestate.csv")
sp <- read.csv("data/realestate-split-folds.csv")
dat <- merge(re, sp, by = "row_id")
preds <- c("date", "age", "distance", "stores", "latitude", "longitude")
train <- dat[dat$split == "train", ]
test <- dat[dat$split == "test", ]
Lambda <- c(0, 10^(-4 + 0.05 * 0:120))
K <- 10
fold_mse <- matrix(NA_real_, K, length(Lambda))

for (k in seq_len(K)) {
  tr <- train[train$cv_fold != k, ]
  va <- train[train$cv_fold == k, ]
  sc <- scale_fit(as.matrix(tr[, preds]), as.matrix(va[, preds]))
  y <- tr$price
  ybar <- mean(y)
  ytilde <- y - ybar
  XtX <- crossprod(sc$Xfit)
  Xty <- crossprod(sc$Xfit, ytilde)
```

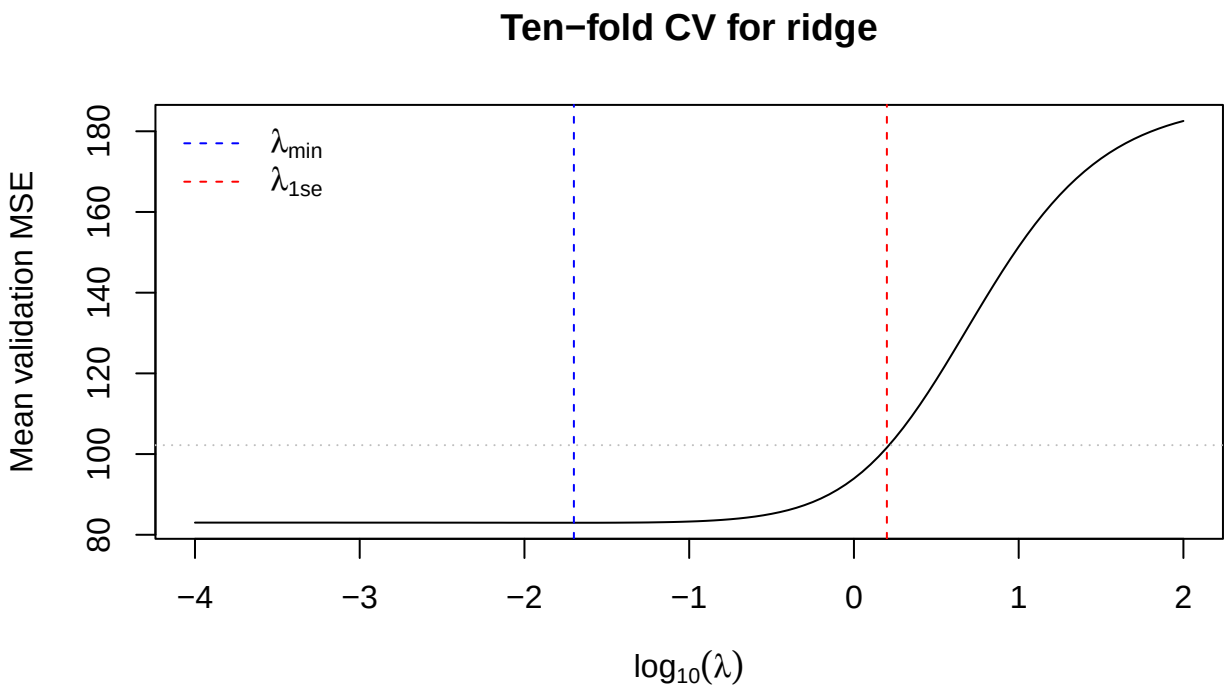
```
  nfit <- nrow(tr)
  p <- ncol(sc$Xfit)
  for (j in seq_along(Lambda)) {
    lam <- Lambda[j]
    b <- if (lam == 0) {
      as.numeric(qr.coef(qr(sc$Xfit), ytilde))
    } else {
      as.numeric(solve(XtX + nfit * lam * diag(p), Xty))
    }
    yhat <- ybar + as.numeric(sc$Xnew %*% b)
    fold_mse[k, j] <- mean((va$price - yhat)^2)
  }
}

mean_mse <- colMeans(fold_mse)
se_mse <- apply(fold_mse, 2, sd) / sqrt(K)
imin <- which.min(mean_mse)
lambda_min <- Lambda[imin]
thresh <- mean_mse[imin] + se_mse[imin]
lambda_1se <- max(Lambda[mean_mse <= thresh + 1e-12])
c(lambda_min = lambda_min,
  cv_min = mean_mse[imin],
  se_min = se_mse[imin],
  lambda_1se = lambda_1se)
```

```
##  lambda_min      cv_min      se_min  lambda_1se
##  0.01995262 82.97764939 19.22991373  1.58489319
```

(a)

```
pos <- Lambda > 0
plot(log10(Lambda[pos]), mean_mse[pos], type = "l",
     xlab = expression(log[10](lambda)),
     ylab = "Mean validation MSE",
     main = "Ten-fold CV for ridge")
abline(v = log10(lambda_min), col = "blue", lty = 2)
abline(v = log10(lambda_1se), col = "red", lty = 2)
abline(h = thresh, col = "gray", lty = 3)
legend("topleft", bty = "n",
       legend = c(expression(lambda[min]), expression(lambda[1*se])),
       col = c("blue", "red"), lty = 2)
```



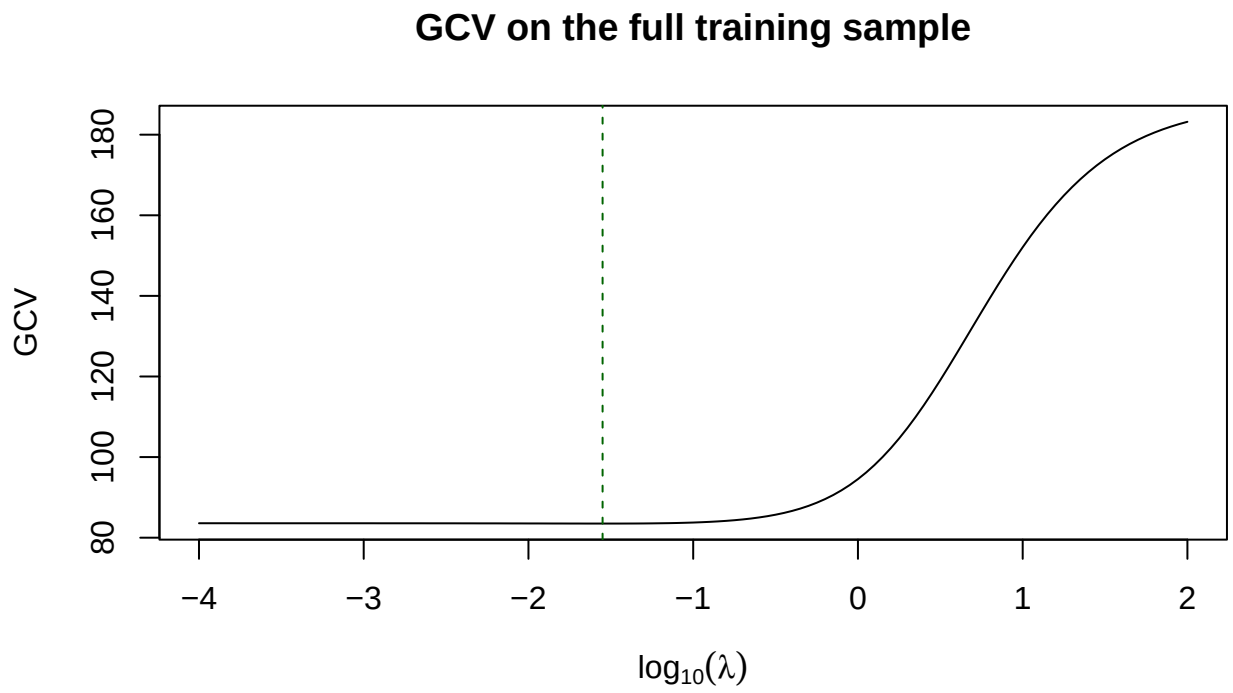
Ten-fold CV gives $\lambda_{\min} = 10^{-1.7} \approx 0.0200$, with mean validation MSE 82.98 and $\mathrm{SE}(\lambda_{\min}) = 19.23$. The one-standard-error cutoff is 102.21. The largest eligible penalty is $\lambda_{1\mathrm{se}} \approx 1.58$.

(b)

```
ntr <- nrow(train)
sc_all <- scale_fit(as.matrix(train[, preds]), as.matrix(train[, preds]))
y <- train$price
ybar <- mean(y)
ytilde <- y - ybar
XtX <- crossprod(sc_all$Xfit)
Xty <- crossprod(sc_all$Xfit, ytilde)
d2 <- svd(sc_all$Xfit, nu = 0, nv = 0)$d^2
p <- ncol(sc_all$Xfit)
gcv <- df_path <- numeric(length(Lambda))
for (j in seq_along(Lambda)) {
  lam <- Lambda[j]
  rho <- if (lam == 0) rep(1, p) else d2 / (d2 + ntr * lam)
  b <- ridge_beta(sc_all$Xfit, ytilde, lam)
  yhat <- ybar + as.numeric(sc_all$Xfit %*% b)
  df_path[j] <- 1 + sum(rho)
  gcv[j] <- mean((y - yhat)^2) / (1 - df_path[j] / ntr)^2
}
ig <- which.min(gcv)
lambda_gcv <- Lambda[ig]
c(lambda_gcv = lambda_gcv, GCV = gcv[ig], df_eff = df_path[ig])
```

```
##  lambda_gcv         GCV      df_eff
##  0.02818383 83.50191402  6.68865672
```

```r
plot(log10(Lambda[pos]), gcv[pos], type = "l",
     xlab = expression(log[10](lambda)),
     ylab = "GCV",
     main = "GCV on the full training sample")
abline(v = log10(lambda_gcv), col = "darkgreen", lty = 2)
```



GCV is minimized at $\lambda_{\text{GCV}} \approx 0.0282$, a little more shrinkage than λ_{min}.

(c)

```r
sc_te <- scale_fit(as.matrix(train[, preds]), as.matrix(test[, preds]))
eval_lam <- function(lam) {
  b <- ridge_beta(sc_te$Xfit, ytilde, lam)
  rho <- if (lam == 0) rep(1, p) else d2 / (d2 + ntr * lam)
  yhat_tr <- ybar + as.numeric(sc_te$Xfit %*% b)
  yhat_te <- ybar + as.numeric(sc_te$Xnew %*% b)
  data.frame(
    lambda = lam,
    df_eff = 1 + sum(rho),
    train_mse = mean((train$price - yhat_tr)^2),
    test_mse = mean((test$price - yhat_te)^2)
```

```
  )
}
rbind(
  OLS = eval_lam(0),
  CV_min = eval_lam(lambda_min),
  CV_1se = eval_lam(lambda_1se),
  GCV = eval_lam(lambda_gcv)
)
```

```
##            lambda   df_eff train_mse test_mse
## OLS    0.00000000 7.000000  80.09392 66.32246
## CV_min 0.01995262 6.773213  80.13543 65.99957
## CV_1se 1.58489319 3.032870 100.37356 81.19981
## GCV    0.02818383 6.688657  80.17125 65.90276
```

OLS has the smallest training MSE, as it must. $\lambda_{\min}$ and λ_{GCV} keep about 6.7–6.8 effective degrees of freedom and similar test MSE (about 66.0 and 65.9). The one-standard-error rule uses only about 3.0 df and has a larger observed test MSE (81.2).

On this split GCV has the smallest test MSE, but that number was looked at after λ was already chosen. Picking a model from this table would reuse the test sample for selection. A different test split could reorder the four test MSEs.

Reference

Yeh, I-Cheng. Real Estate Valuation Data Set. UCI Machine Learning Repository.

`explain-to-me` skill

```
---
name: explain-to-me
description: >
  Explain a STAT 432 homework question without solving it.
  Use only when the student explicitly asks to use
  explain-to-me on a question.
---

# Explain a homework question

Help the student understand what a homework question is asking.
Do not compute answers, write solution code, or fill in the report.

When asked to use this skill on a question:

1. State the goal in one or two sentences.
2. List the required outputs (numbers, plots, derivations,
   or explanations).
3. Note any constraints: seeds, scaling rules, which rows
   or files to use, and what must stay unused while tuning.
```

4. Point to the matching Week 3 lecture:
   [Ridge Regression: Stability Through Shrinkage](
   https://teazrq.github.io/stat432rpy/topics/ridge-regression/
   ridge-regression.html)
   or *[From a Penalized Objective to a Fitted Ridge Model]*(
   https://teazrq.github.io/stat432rpy/topics/ridge-regression/
   optimization-and-cross-validation.html).
5. Stop there. Leave the solution to the student.